

Data Structures

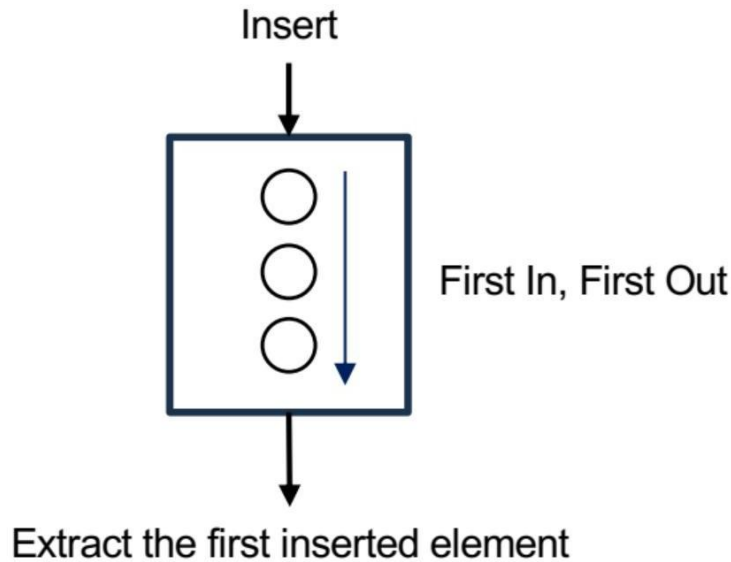
Lecture 17: Heap Sort

Soobin Um

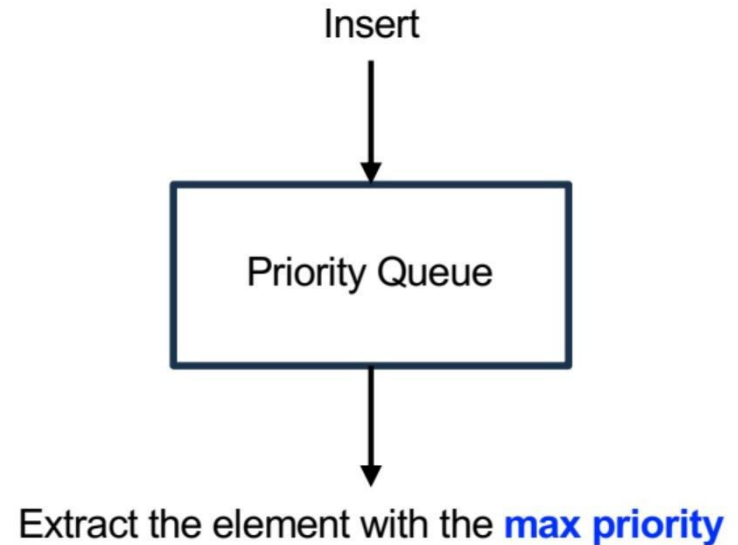
May 13, 2026

Recap: Priority Queue (우선순위 큐)

- 각 원소가 우선순위 값을 가지며, 이 우선순위에 따라 원소들이 저장/처리되는 큐의 한 종류
- FIFO 구조인 일반 큐와 달리, 우선순위가 높은 원소가 낮은 원소보다 먼저 처리(dequeue)됨



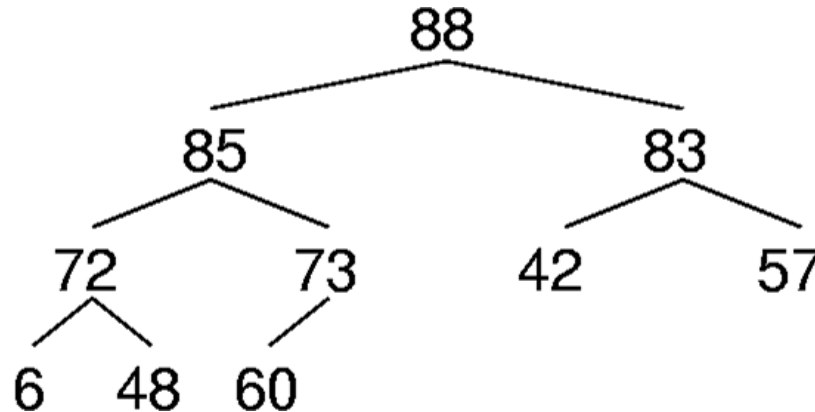
일반적인 큐



우선순위 큐

Recap: Heap (힙: 더미, 무더기)

- 힙 속성(Heap Property)을 만족하는 완전이진트리(Complete BT)
 - 힙 속성
 - Max-heap: 모든 노드는 자손 노드보다 큰 값을 가짐 (→ 본 강의의 초점)
 - Min-heap: 모든 노드는 자손 노드보다 작은 값을 가짐



Max heap의 예

Heap Sort

Heap Sort

- 힙 자료 구조를 이용한 선택 정렬의 개선 버전
 - 선택 정렬(Selection Sort): 매 단계마다 전체 배열에서 최대값 (또는 최소값) 탐색
 - 힙 정렬: 힙 구조를 이용해 최대값(또는 최소값)을 효율적으로 선택

Heap Sort

- 힙 자료 구조를 이용한 선택 정렬의 개선 버전
 - 선택 정렬(Selection Sort): 매 단계마다 전체 배열에서 최대값 (또는 최소값) 탐색
 - 힙 정렬: 힙 구조를 이용해 최대값(또는 최소값)을 효율적으로 선택

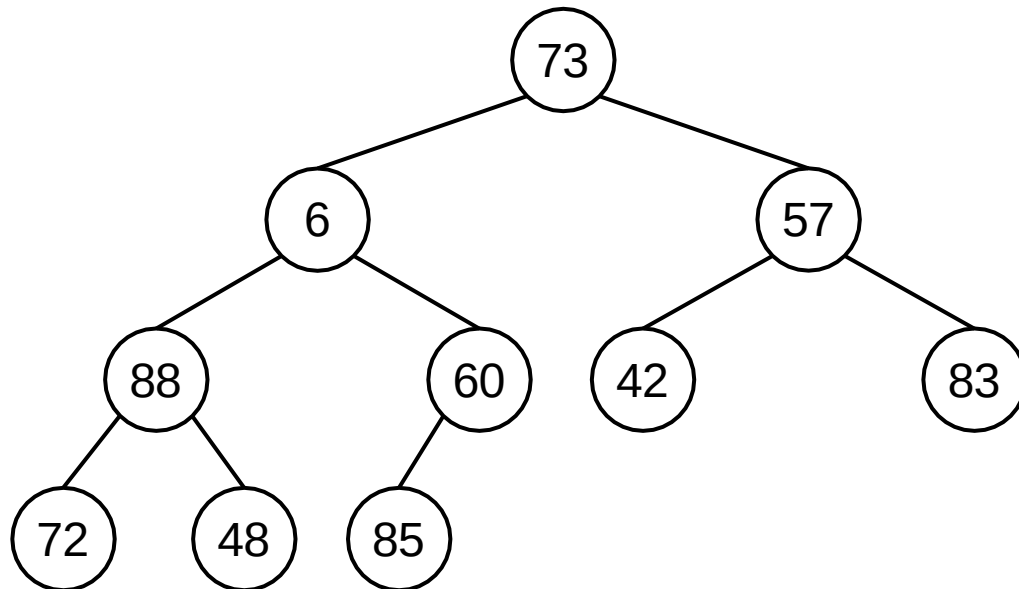
		Selection Sort	Heap Sort
원리		매번 전체 배열에서 최대값 선택	힙의 루트 (최대값) 선택
입력 구조		단순 배열	완전이진트리 (힙)
시간 복잡도	최대값 찾기	$O(n)$	$O(1)$
	재정렬	$O(1)$	$O(\log n)$
	전체	$O(n^2)$	$O(n \log n)$
메모리 사용		추가 메모리 사용 없음	추가 메모리 사용 없음 (in-place)

Heap Sort 과정

1. 크기 n 의 배열(리스트)을 Max-heap으로 변환 ($\rightarrow$ buildHeap)
2. 힙의 루트(최대값)를 배열의 끝으로 이동
3. 힙 크기를 1 줄이고, 루트에서부터 힙 속성 복구 ($\rightarrow$ siftDown)
4. 위 1-3 과정을 n 번 반복

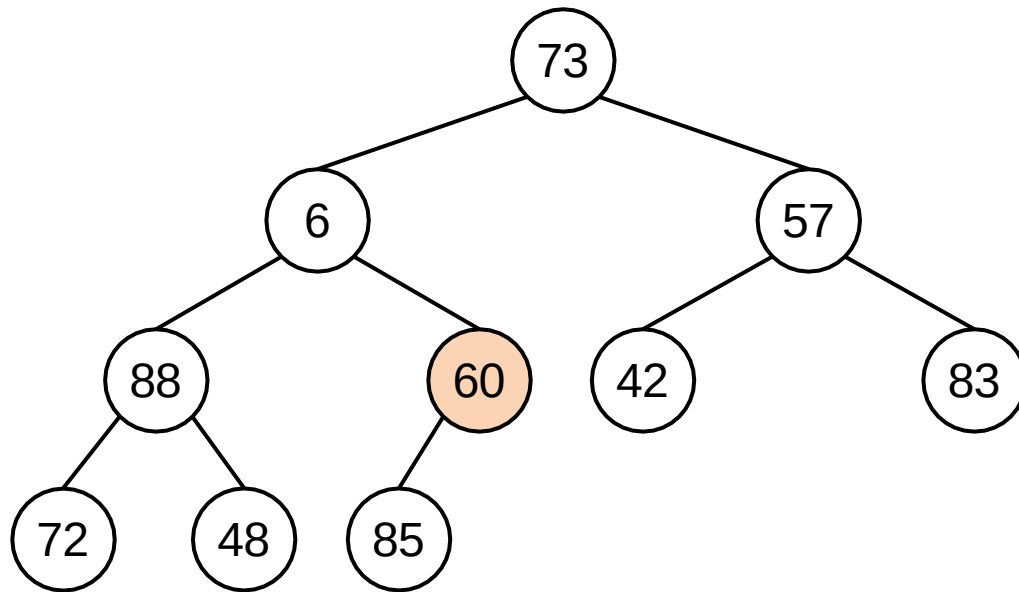
정렬 대상 배열

73	6	57	88	60	42	83	72	48	85
----	---	----	----	----	----	----	----	----	----



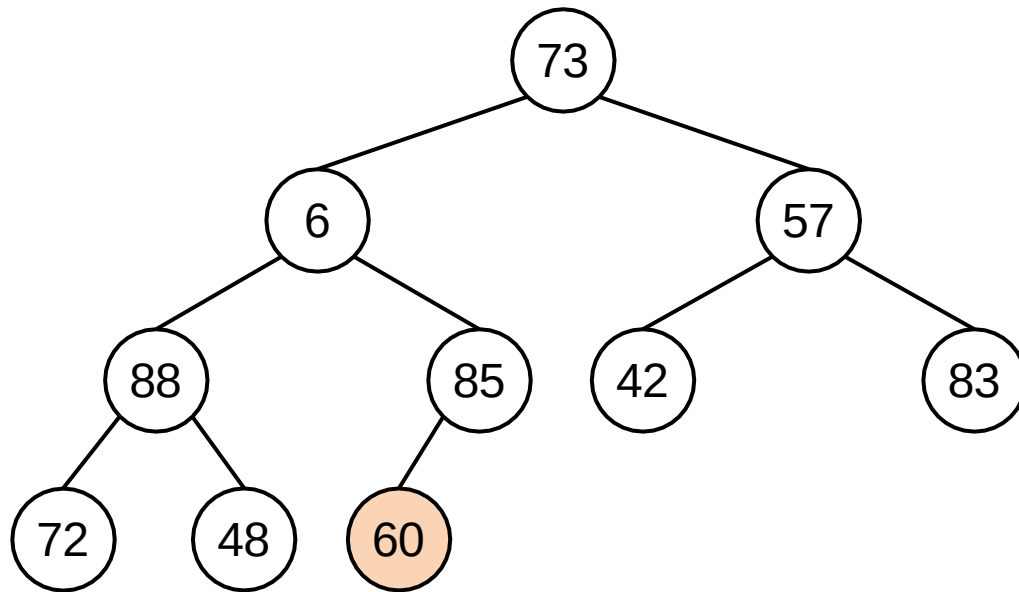
Build Heap

- 마지막 내부 노드에서부터 시작

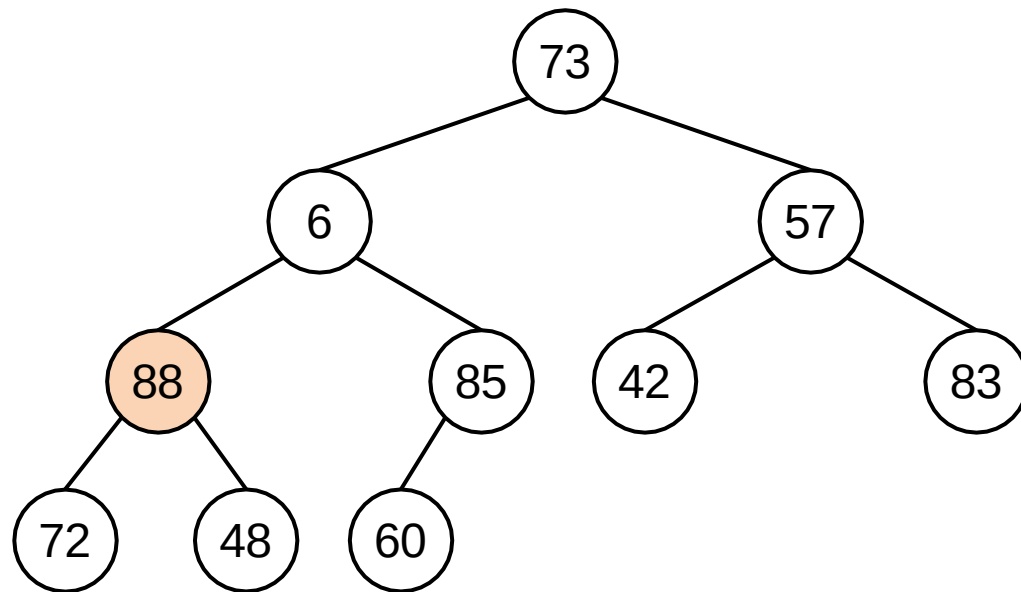


Build Heap

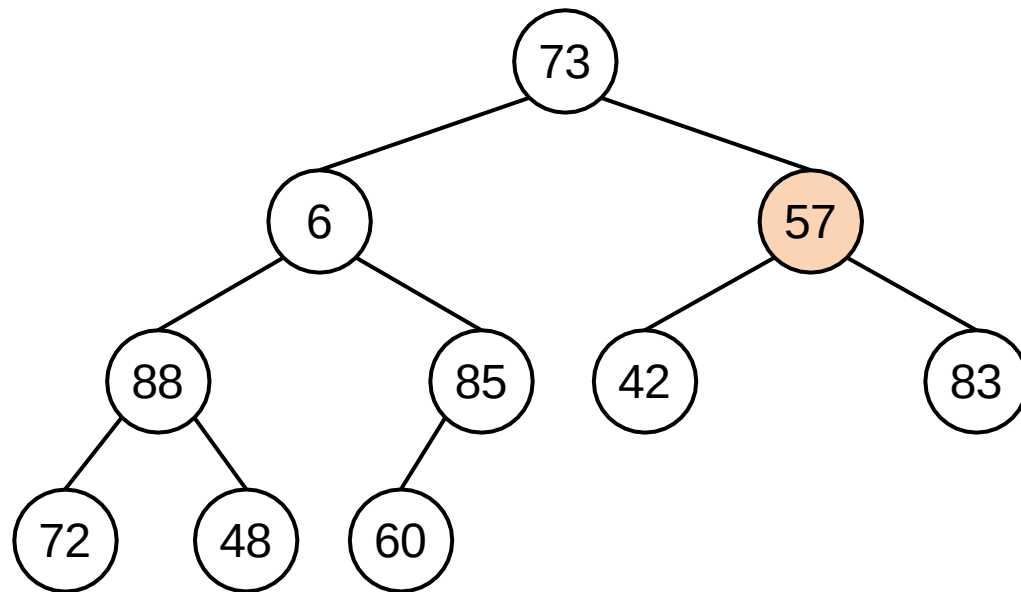
- 자식 노드보다 작을 경우 swap 발생



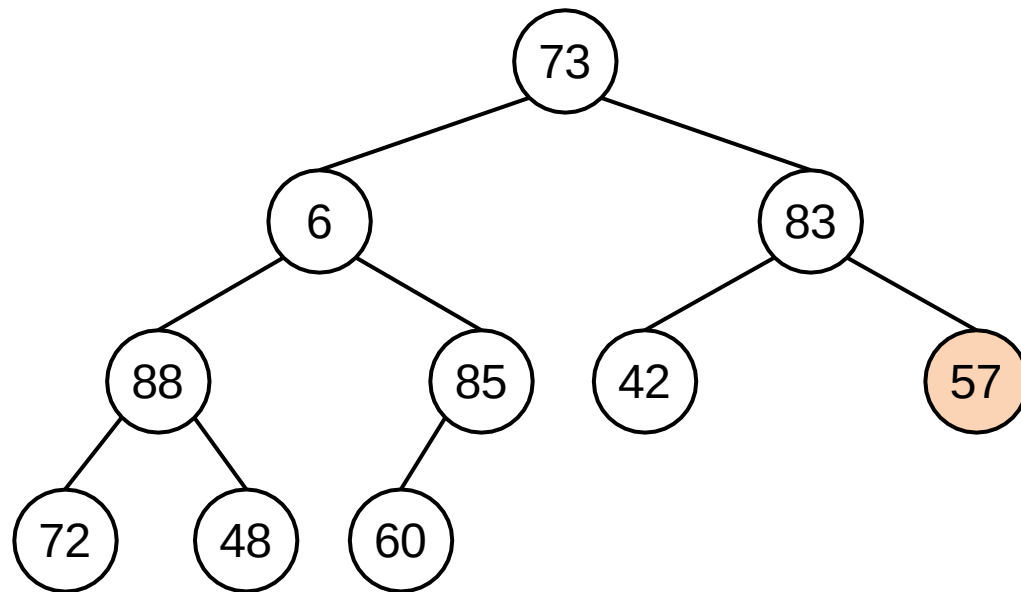
Build Heap



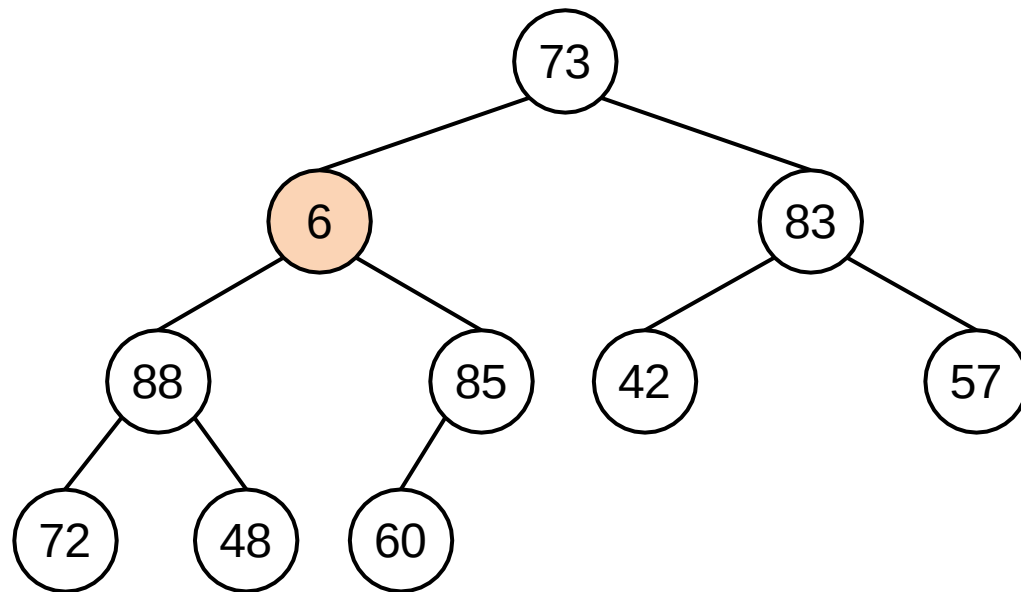
Build Heap



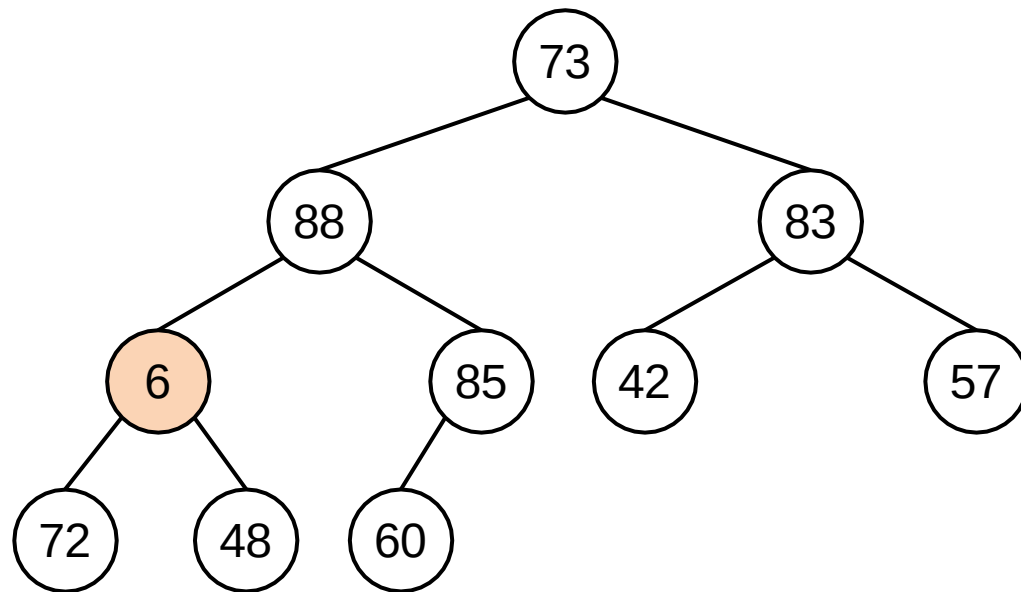
Build Heap



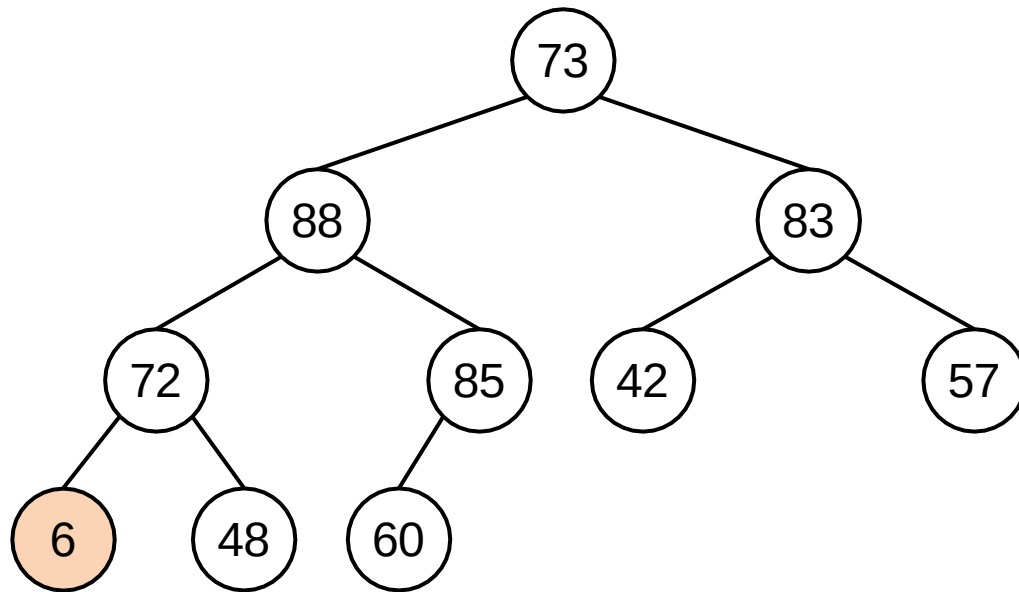
Build Heap



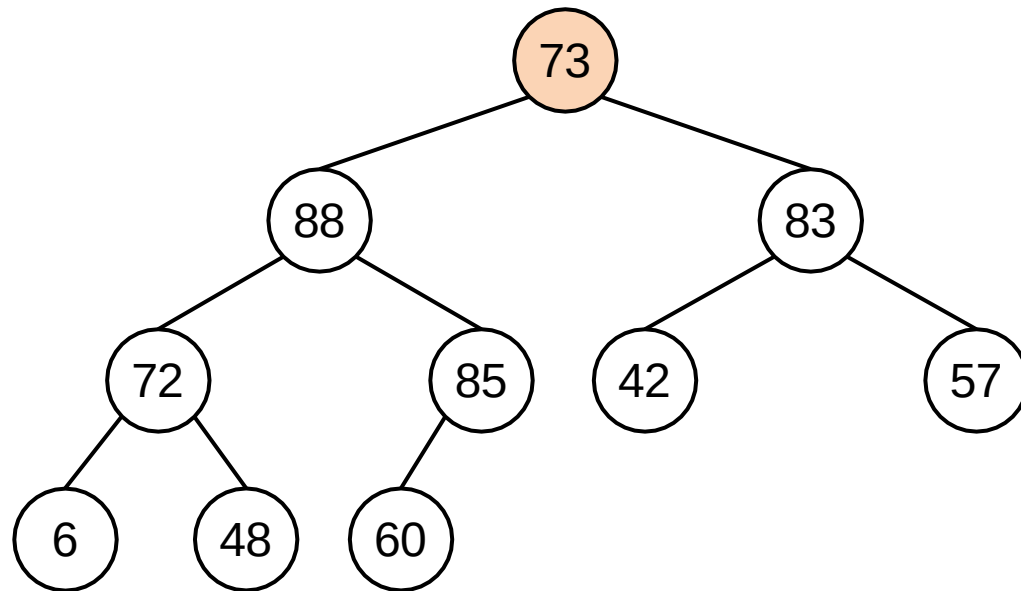
Build Heap



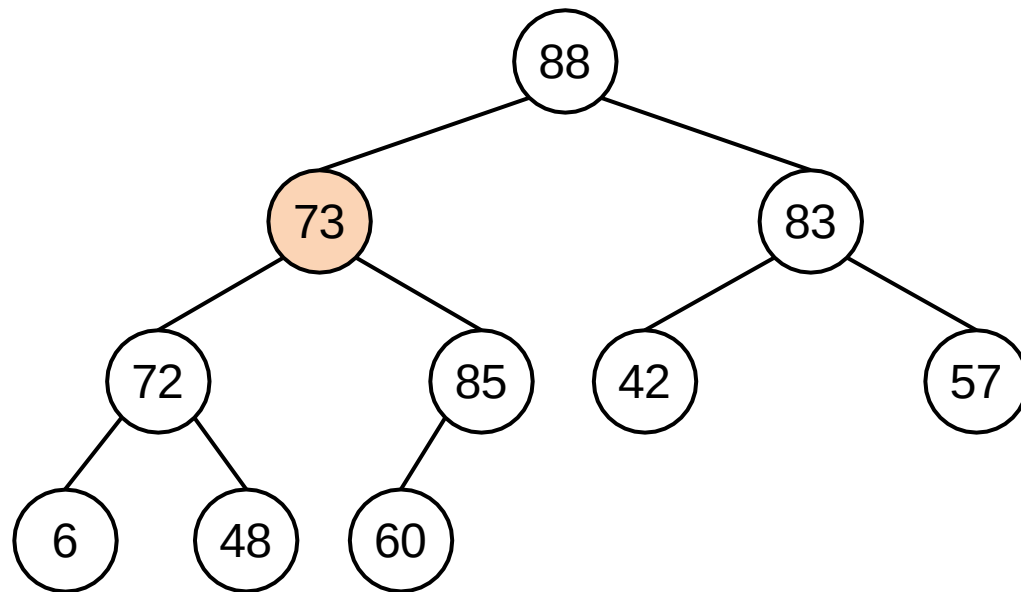
Build Heap



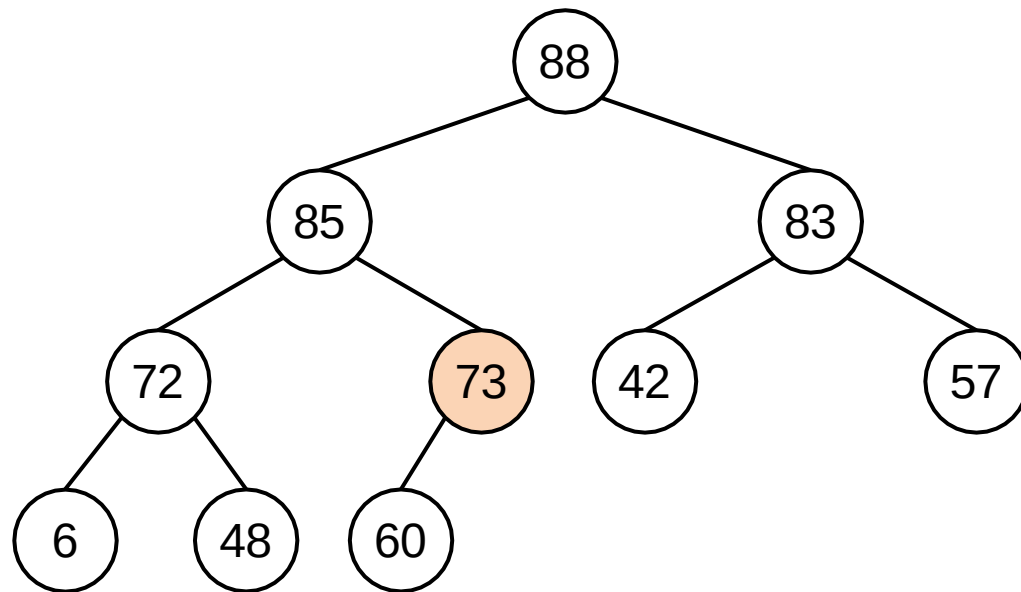
Build Heap



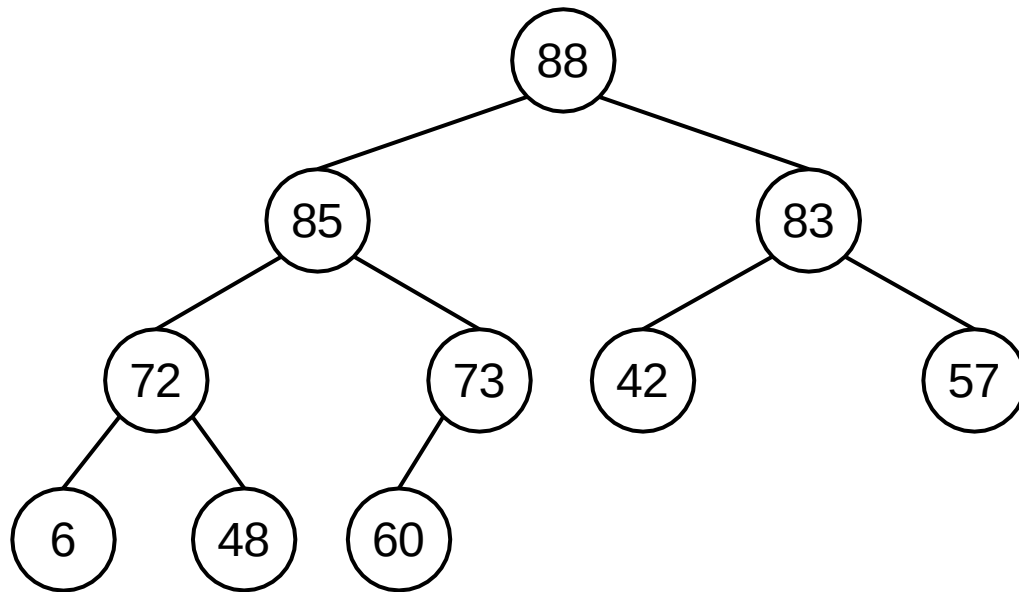
Build Heap



Build Heap



Build Heap: 완성



Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

$$T(n) = \sum_{h=0}^{\log n} 2^h (\log n - h)$$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

$$T(n) = \sum_{h=0}^{\log n} 2^h (\log n - h)$$

$$= n \cdot \sum_{k=0}^{\log n} \frac{k}{2^k}$$

$$(k := \log n - h)$$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

$$\begin{aligned} T(n) &= \sum_{h=0}^{\log n} 2^h (\log n - h) \\ &= n \cdot \sum_{k=0}^{\log n} \frac{k}{2^k} \leq n \cdot \sum_{k=0}^{\infty} \frac{k}{2^k} \\ &\quad (k := \log n - h) \end{aligned}$$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

$$\begin{aligned} T(n) &= \sum_{h=0}^{\log n} 2^h (\log n - h) \\ &= n \cdot \sum_{k=0}^{\log n} \frac{k}{2^k} \leq n \cdot \sum_{k=0}^{\infty} \frac{k}{2^k} = 2n \\ &\quad (k := \log n - h) \end{aligned}$$

Build Heap: 시간 복잡도 $O(n)$

레벨	노드 개수	최대 sift down 횟수
0 (루트)	1	$\log n$
1	2	$\log n - 1$
2	4	$\log n - 2$
...	...	...
$\log n$	$2^{\log n}$	0

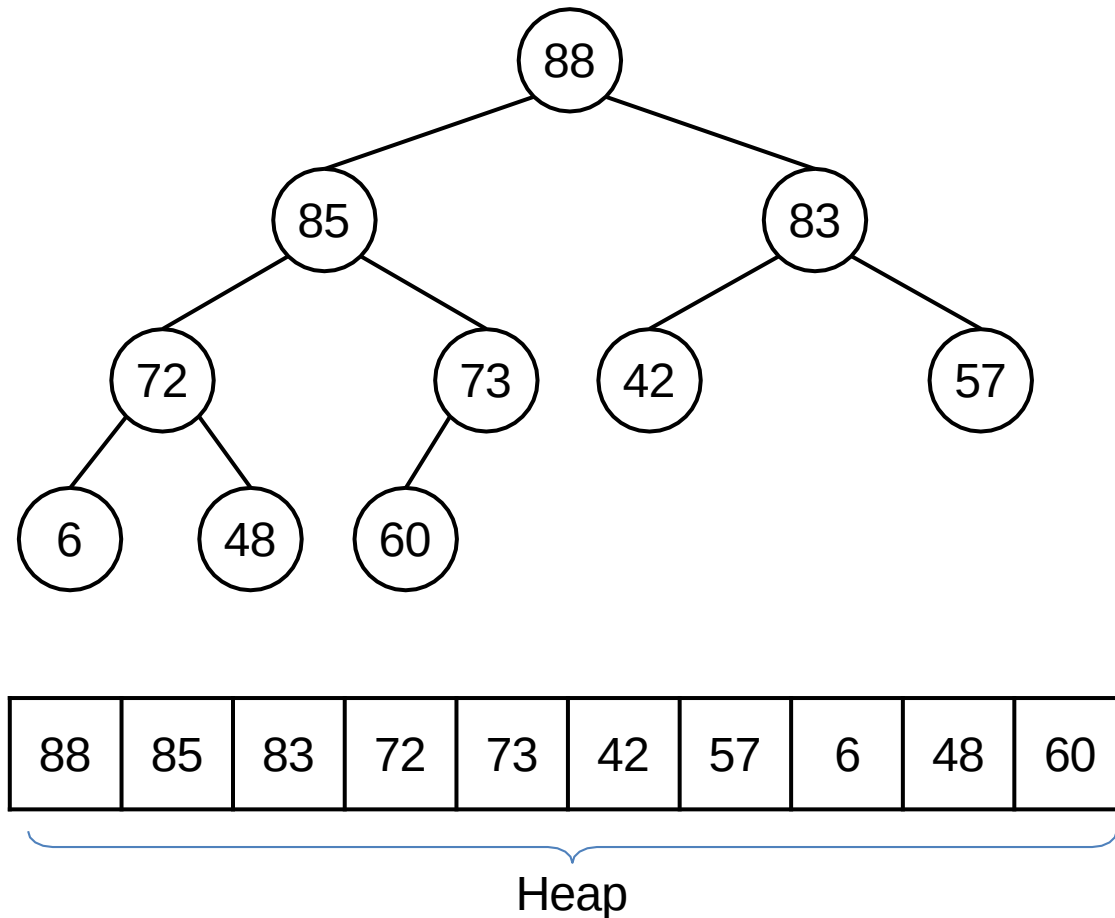
총 수행 비용 ($=T(n)$): 레벨에 따른 (노드 개수 $\times$ 최대 sift down 횟수)의 합

$$\begin{aligned} T(n) &= \sum_{h=0}^{\log n} 2^h (\log n - h) \\ &= n \cdot \sum_{k=0}^{\log n} \frac{k}{2^k} \leq n \cdot \sum_{k=0}^{\infty} \frac{k}{2^k} = 2n \end{aligned} \quad \Rightarrow T(n) \in O(n)$$

$(k := \log n - h)$

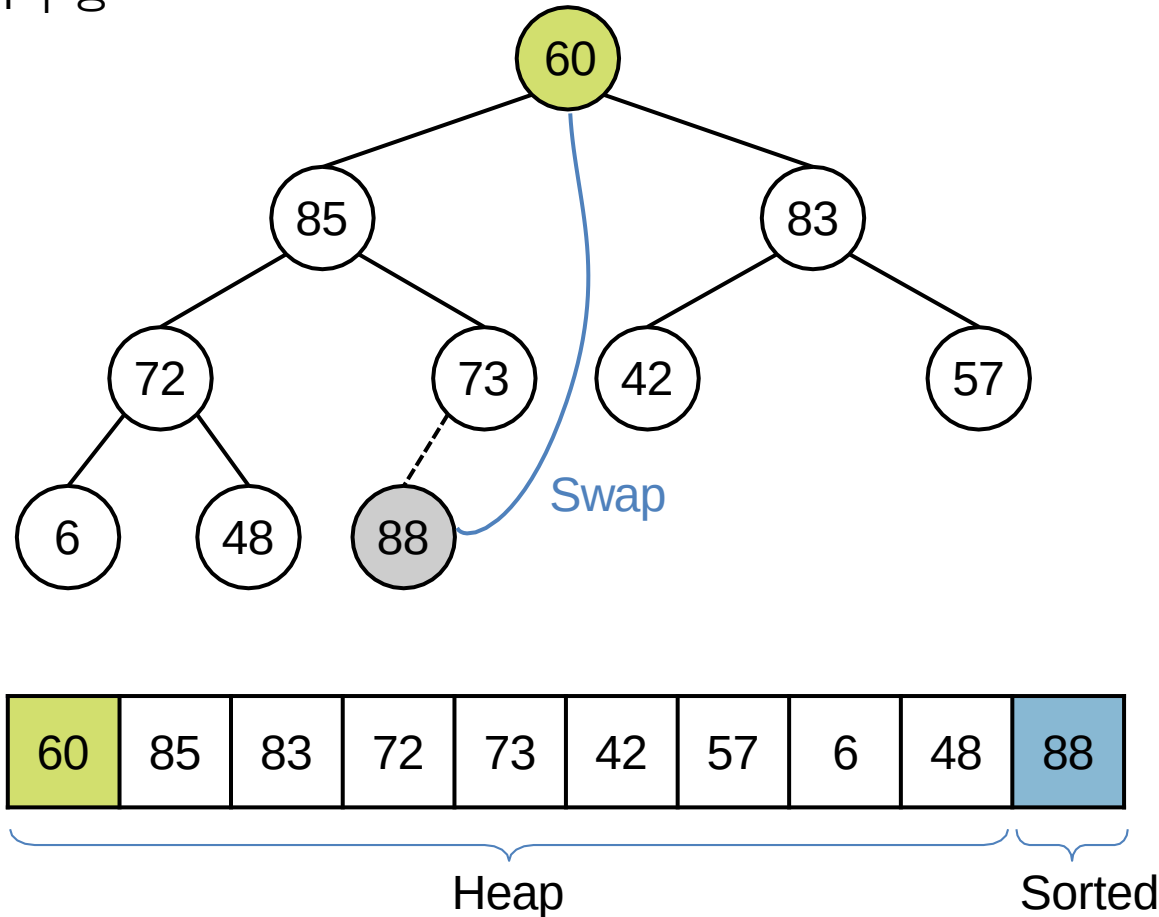
Heap Sort

- 완성되어 있던 힙을 이용하여 시작



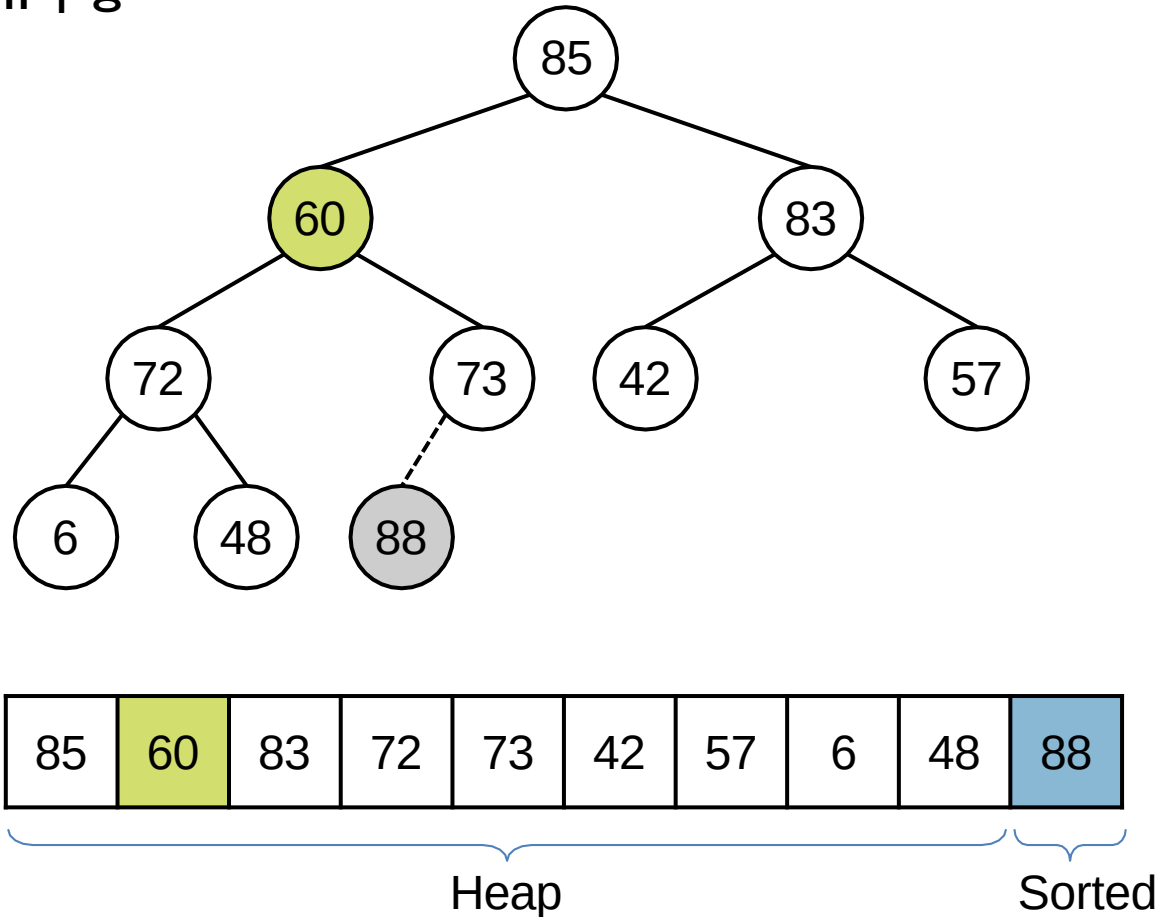
Heap Sort

- 루트 (88) 삭제
 - 루트와 마지막 노드를 서로 교환
 - Sift down 수행



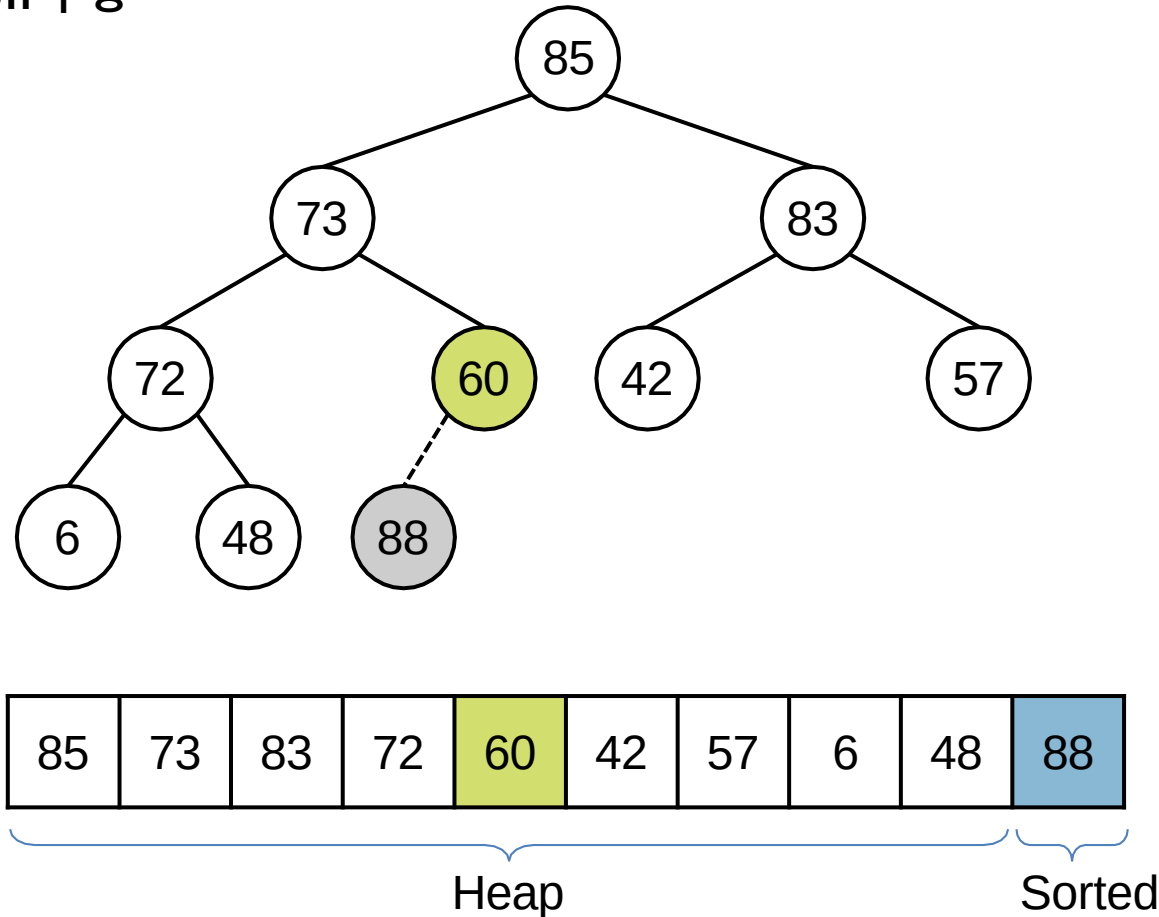
Heap Sort

- 루트 (88) 삭제
 - 루트와 마지막 노드를 서로 교환
 - Sift down** 수행



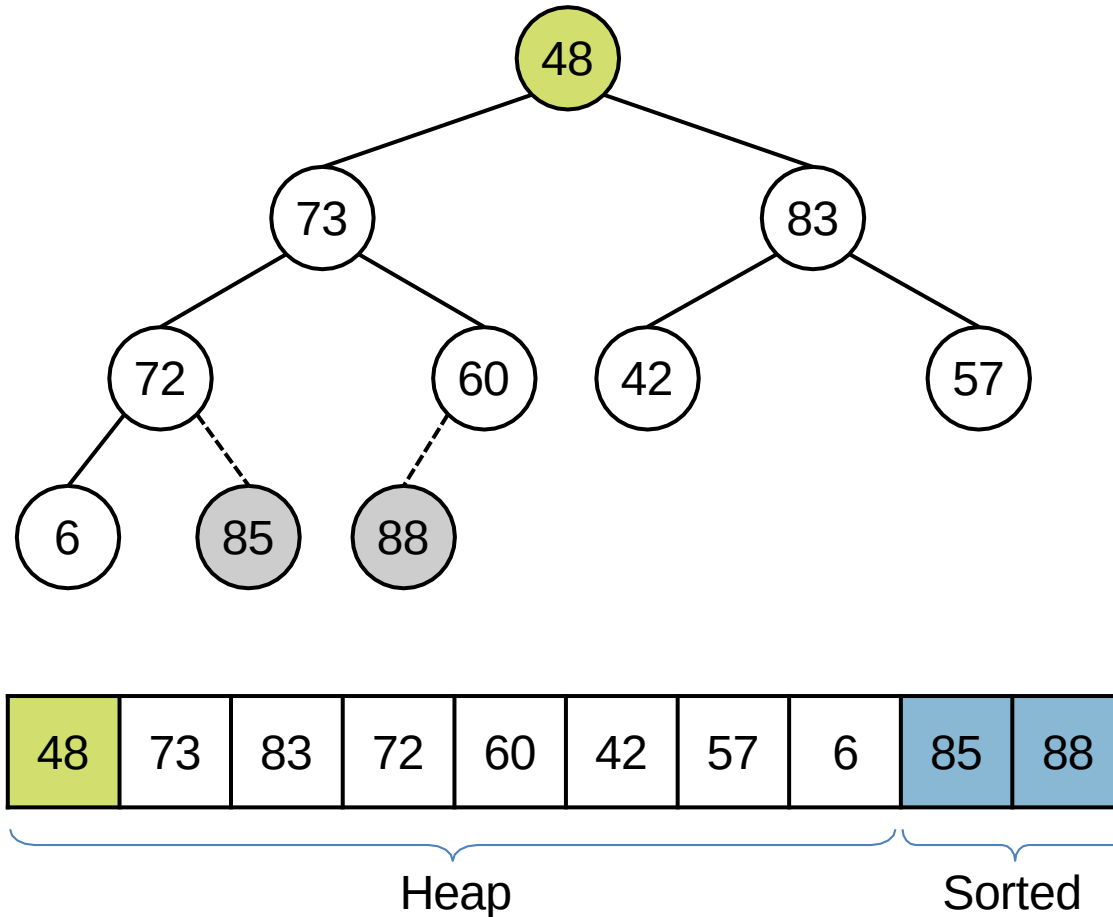
Heap Sort

- 루트 (88) 삭제
 - 루트와 마지막 노드를 서로 교환
 - Sift down** 수행

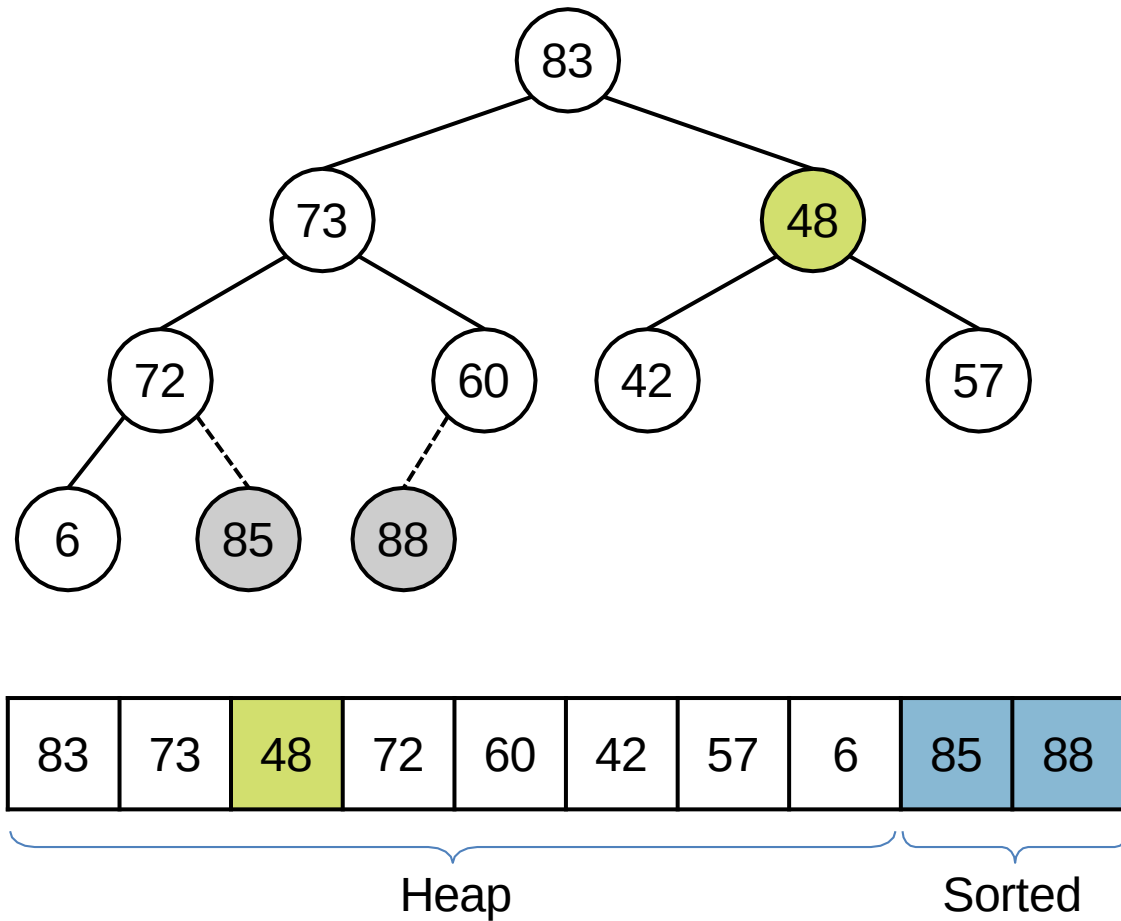


Heap Sort

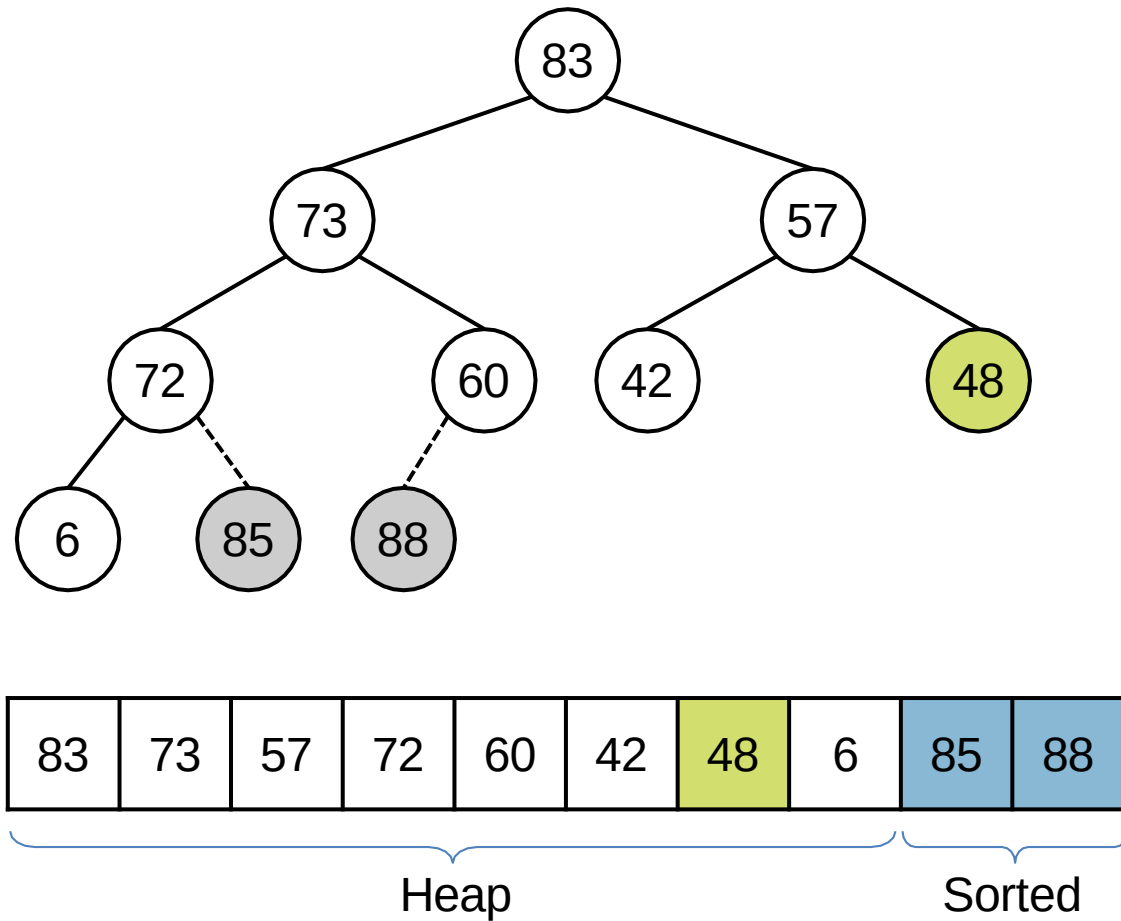
- 루트 (85) 삭제



Heap Sort

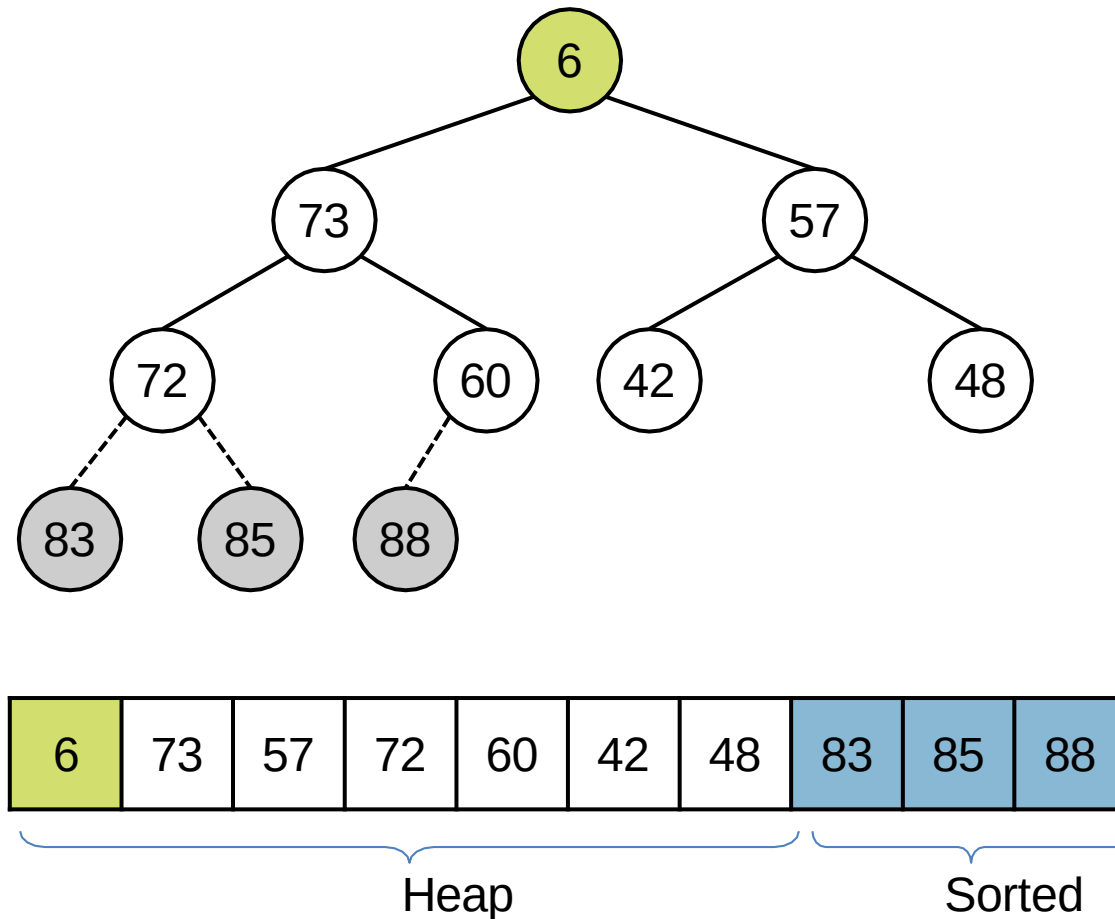


Heap Sort

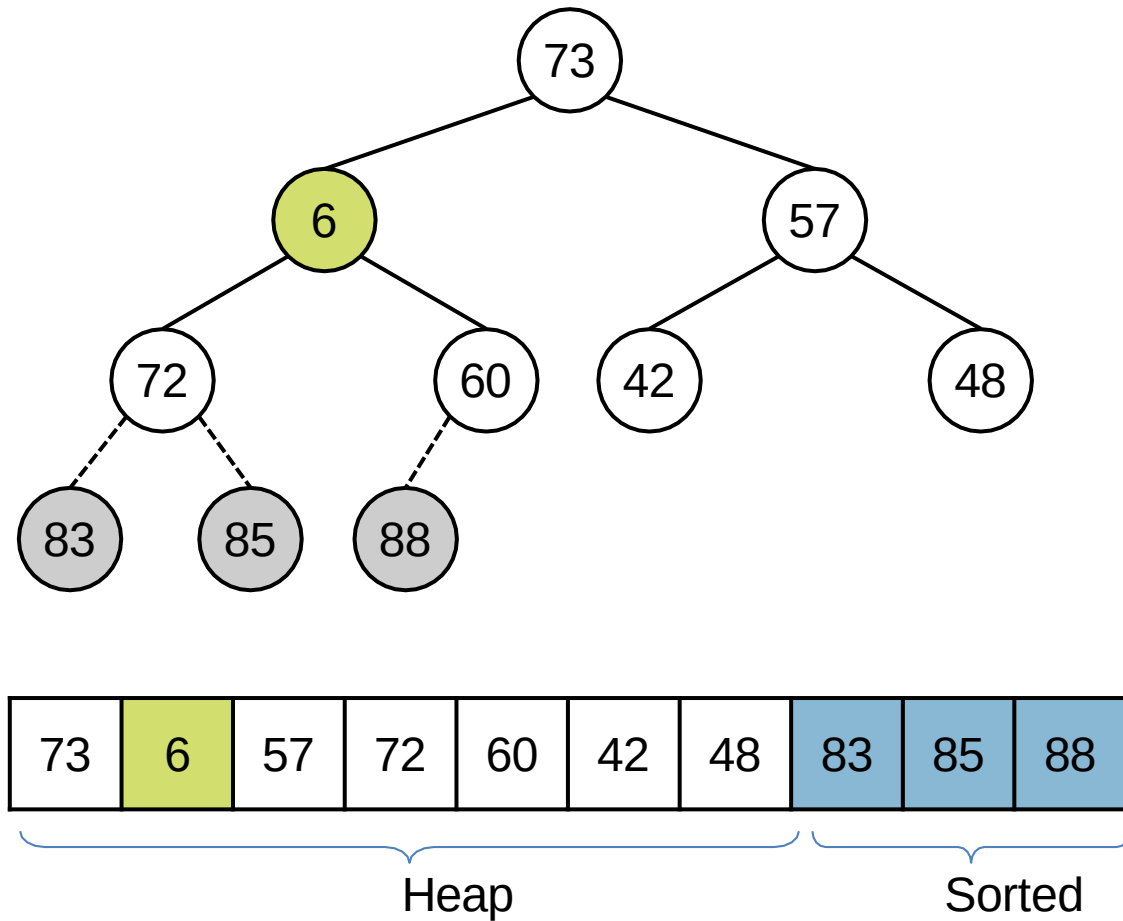


Heap Sort

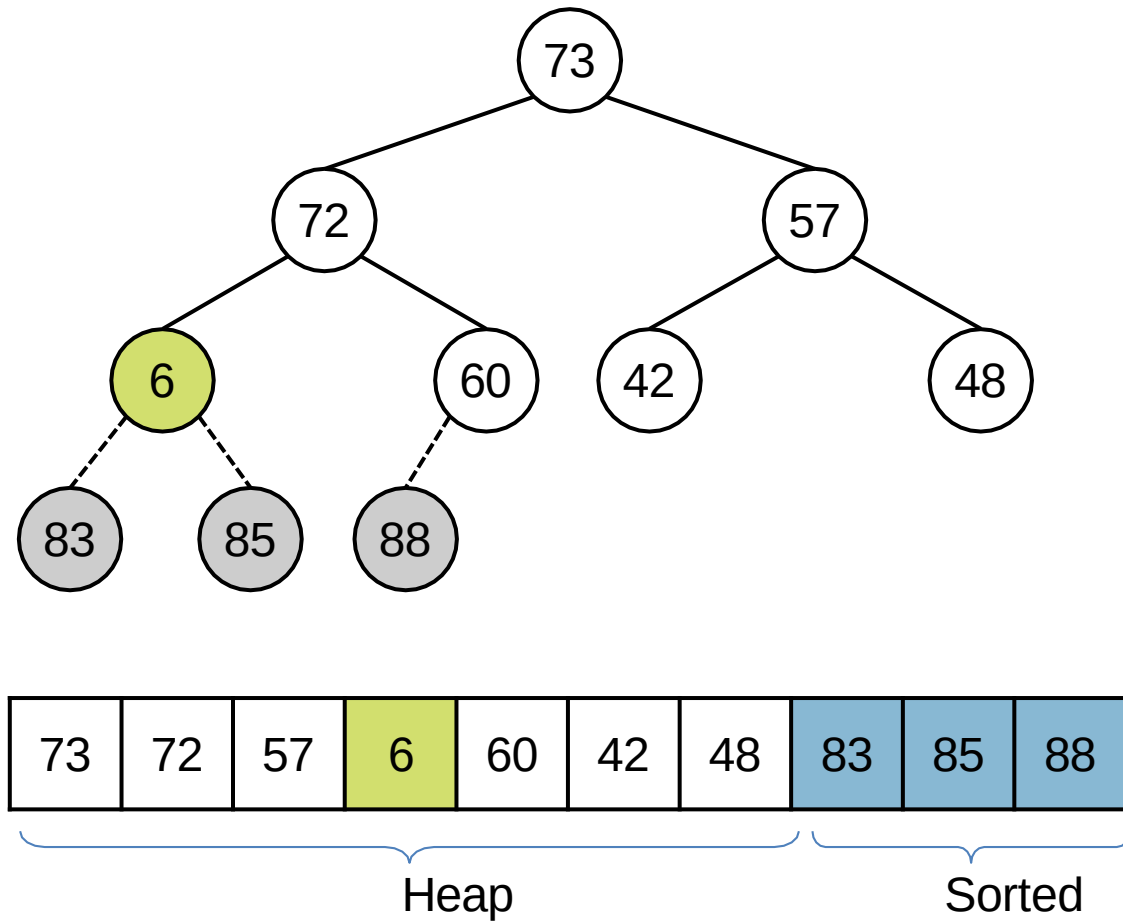
- 루트 (83) 삭제



Heap Sort

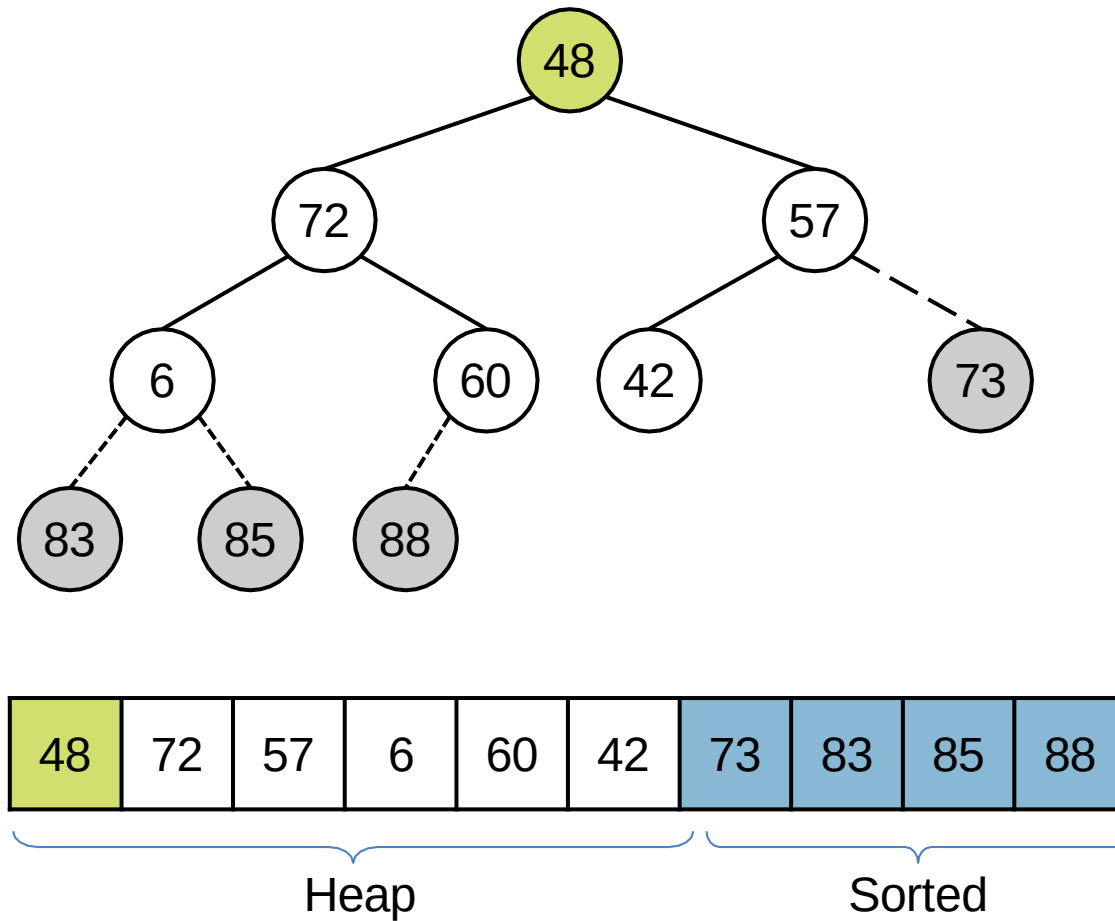


Heap Sort

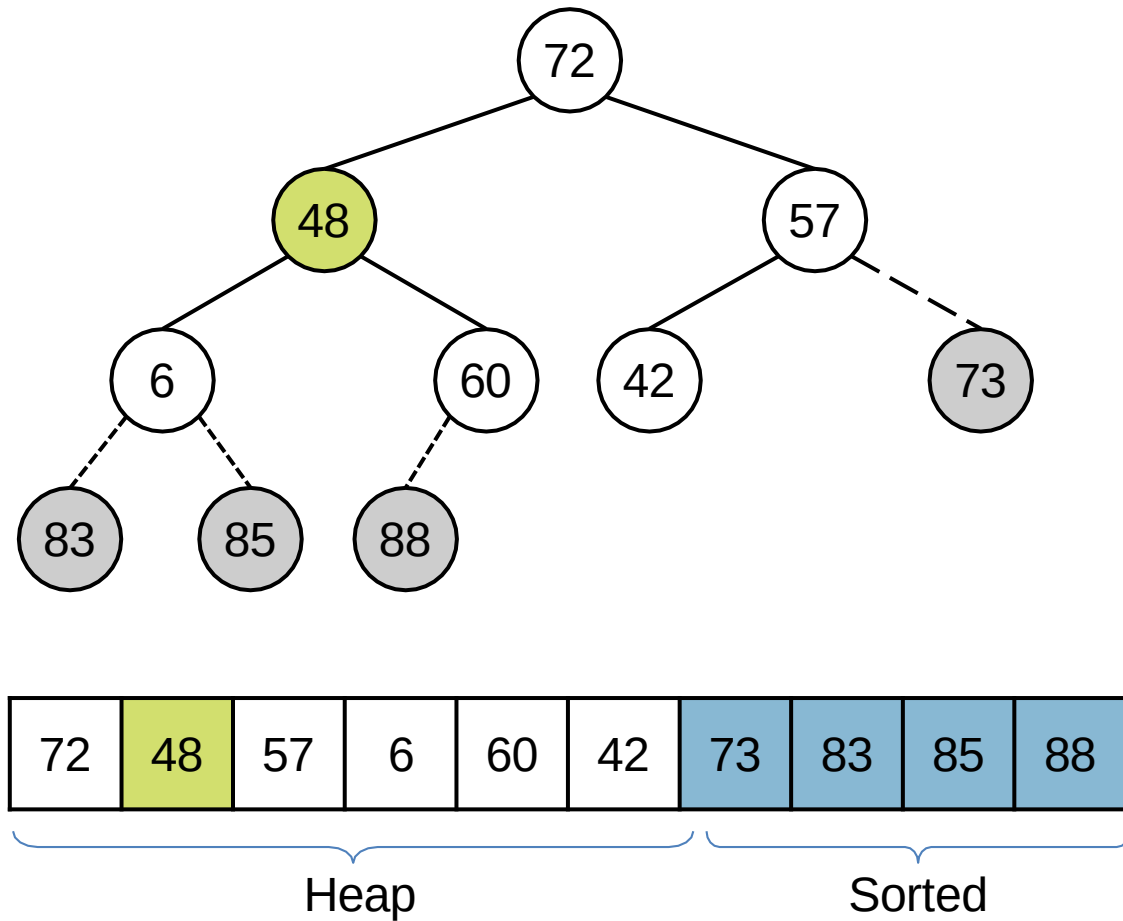


Heap Sort

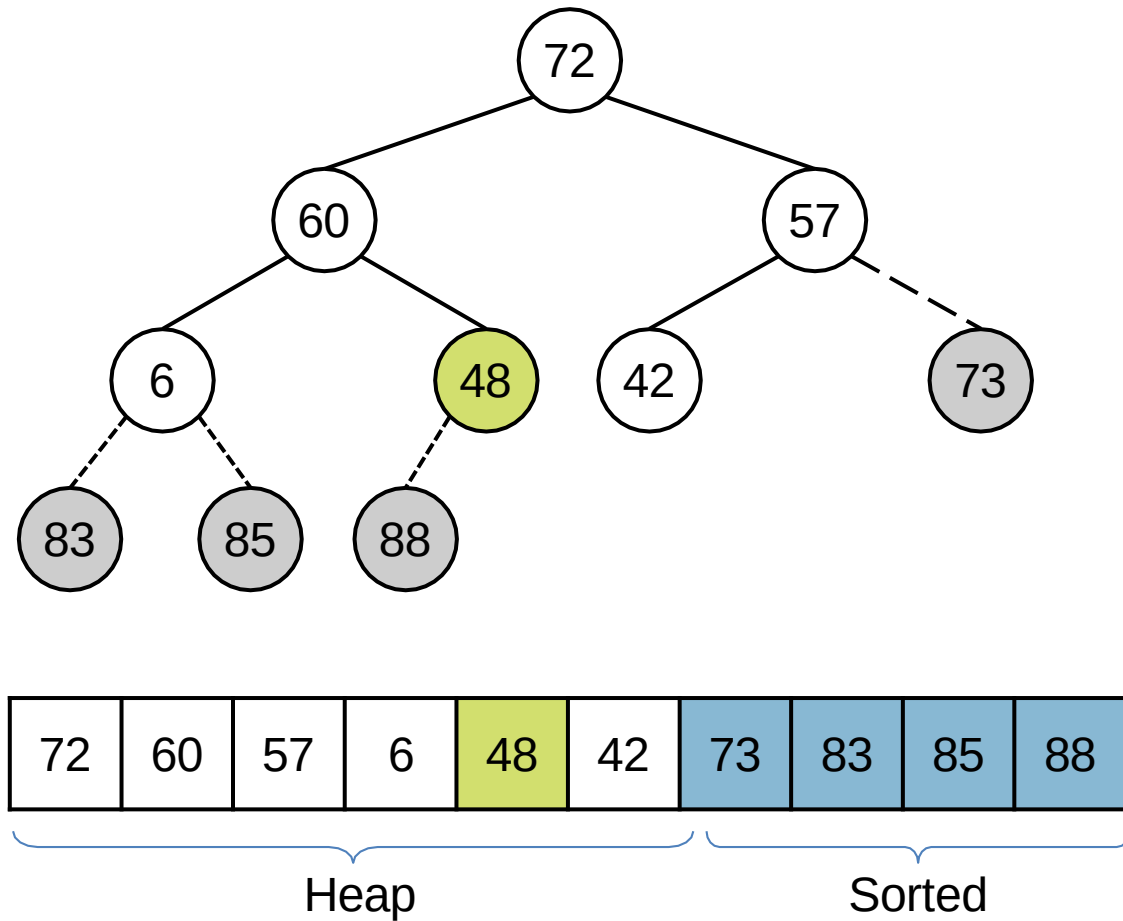
- 루트 (73) 삭제



Heap Sort

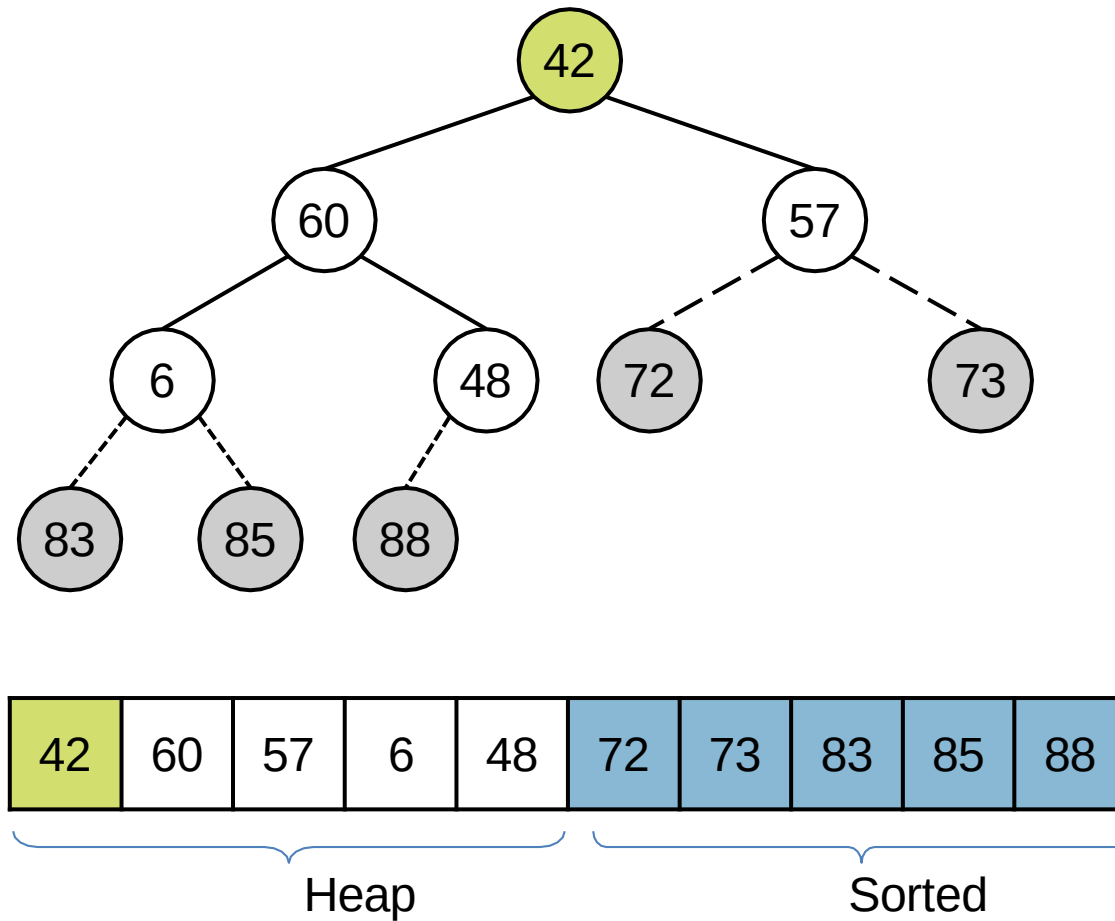


Heap Sort

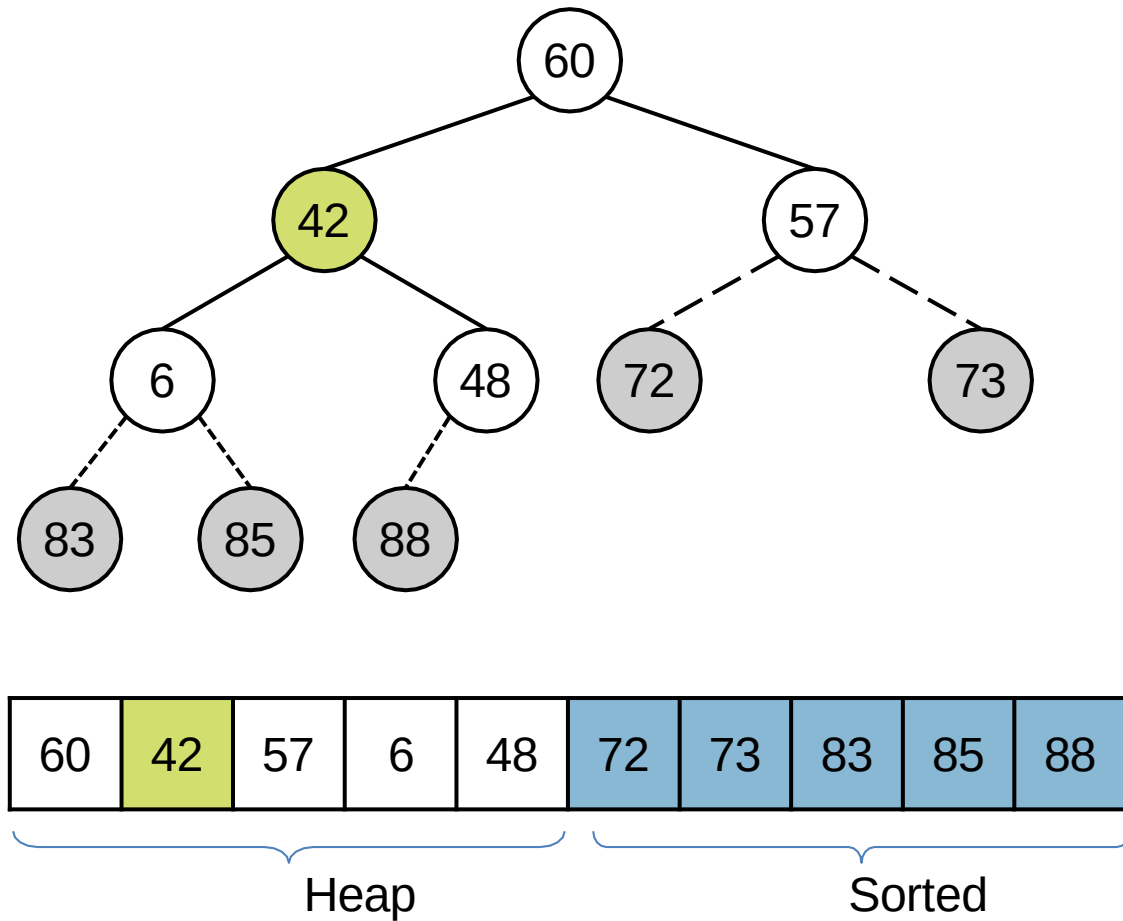


Heap Sort

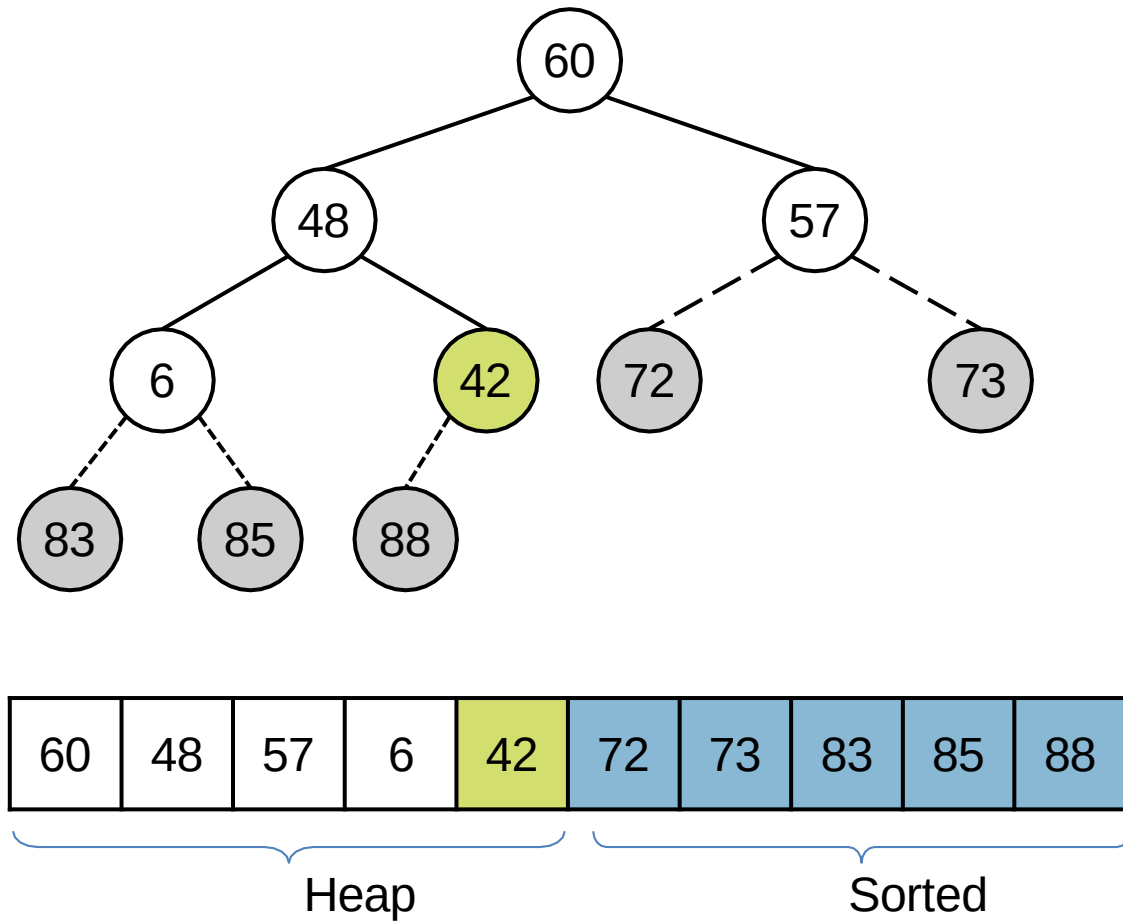
- 루트 (72) 삭제



Heap Sort

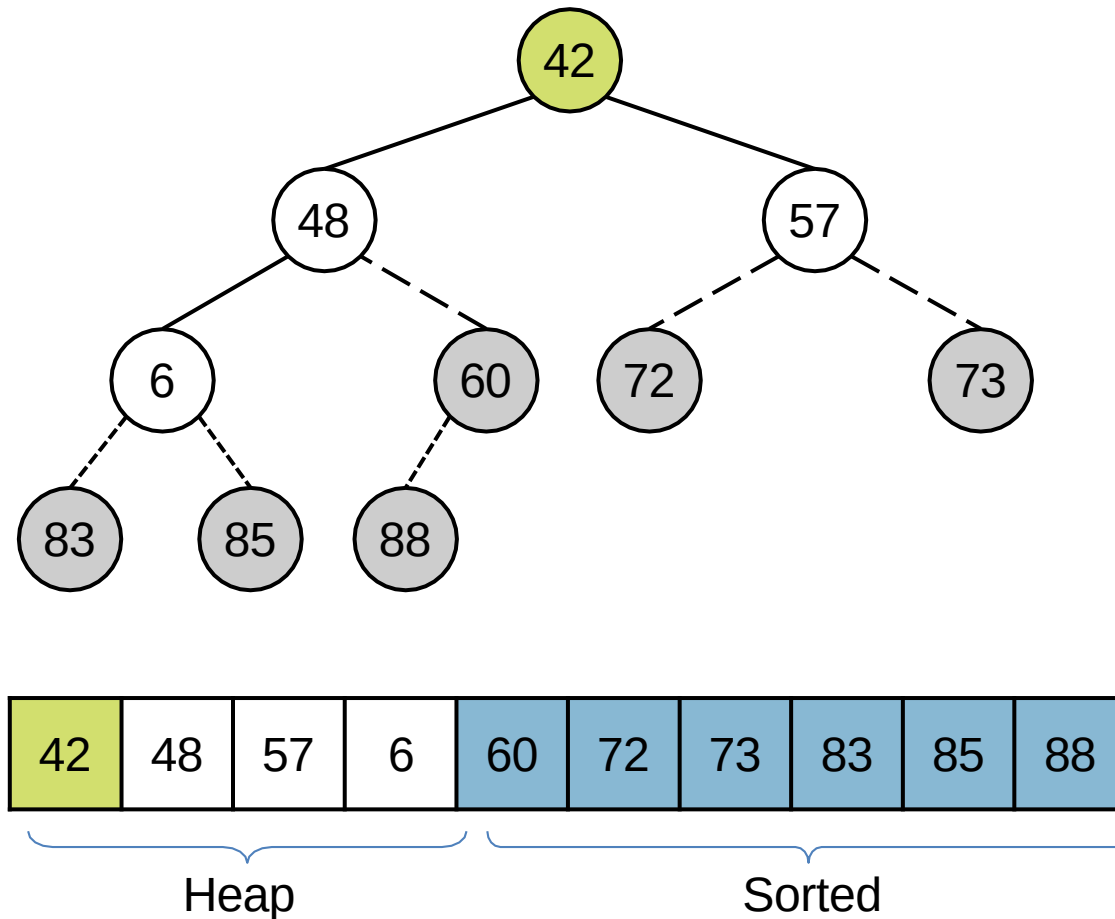


Heap Sort

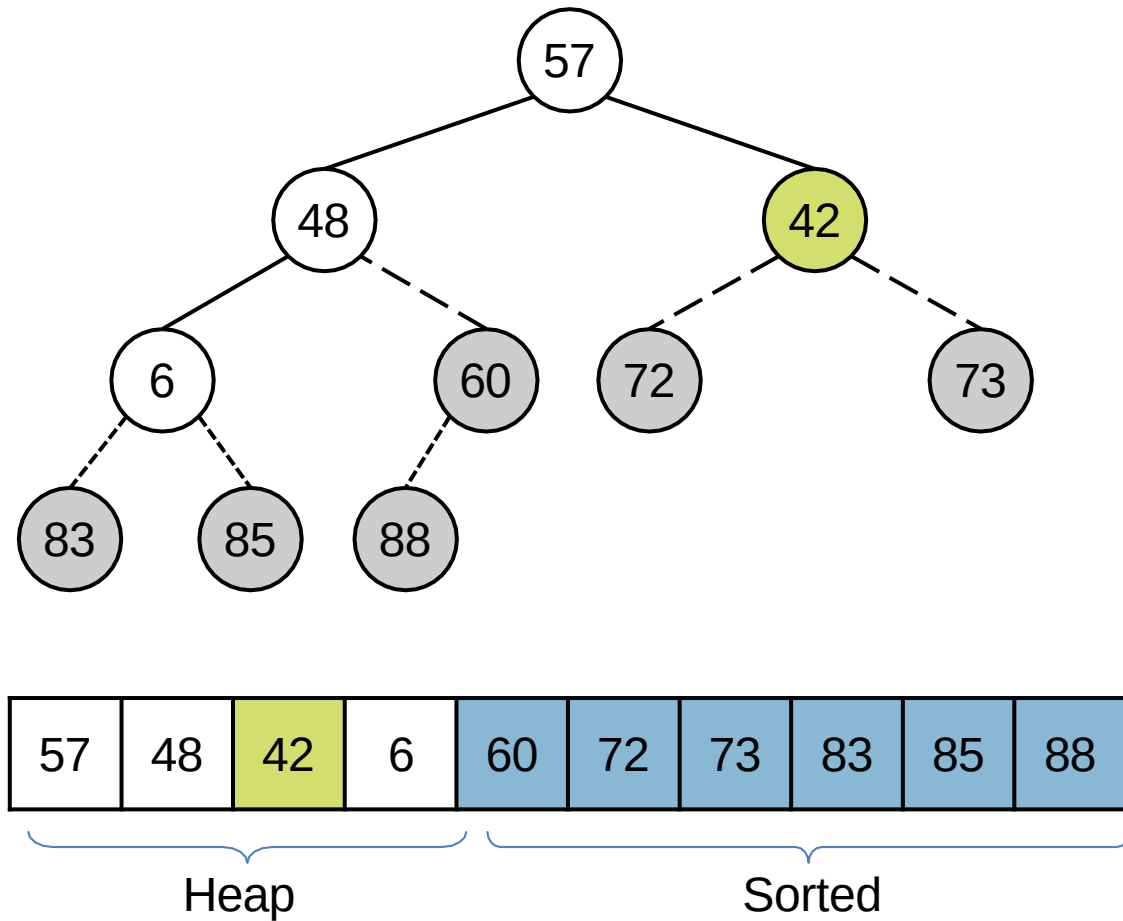


Heap Sort

- 루트 (60) 삭제

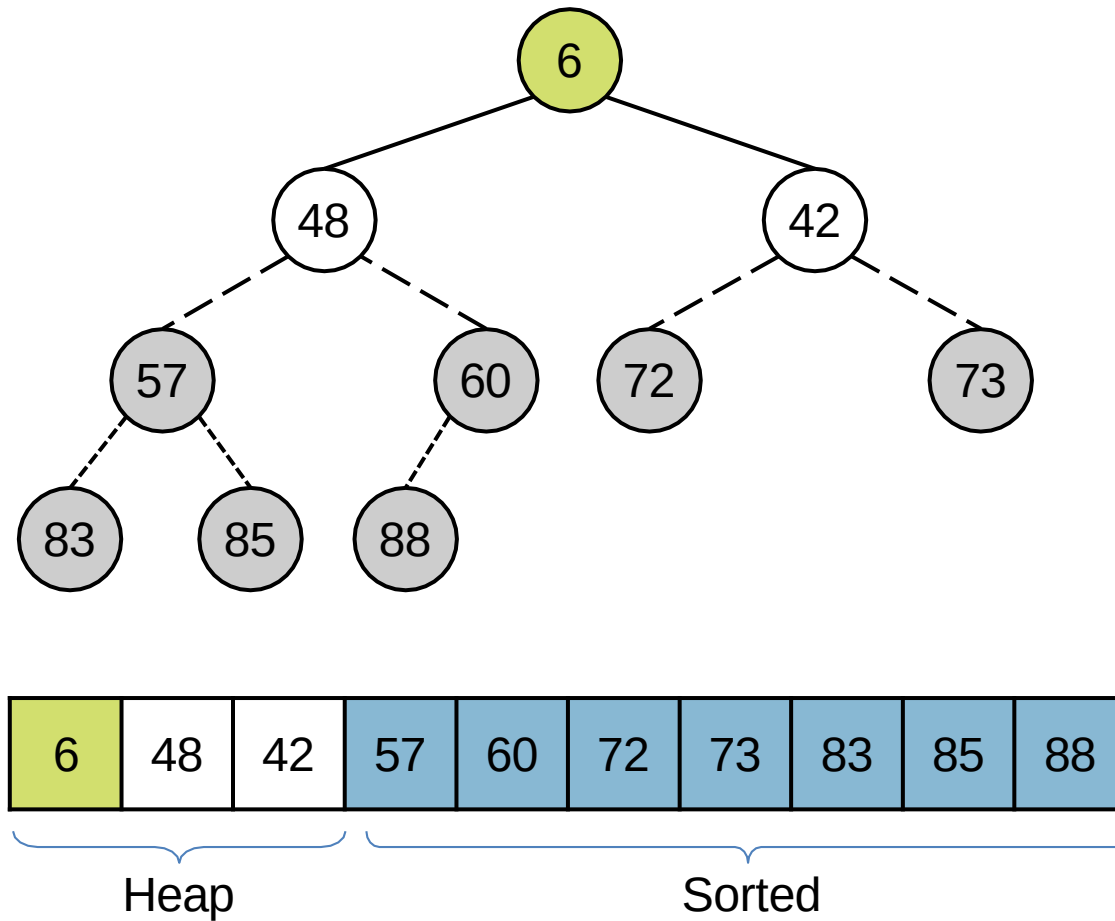


Heap Sort

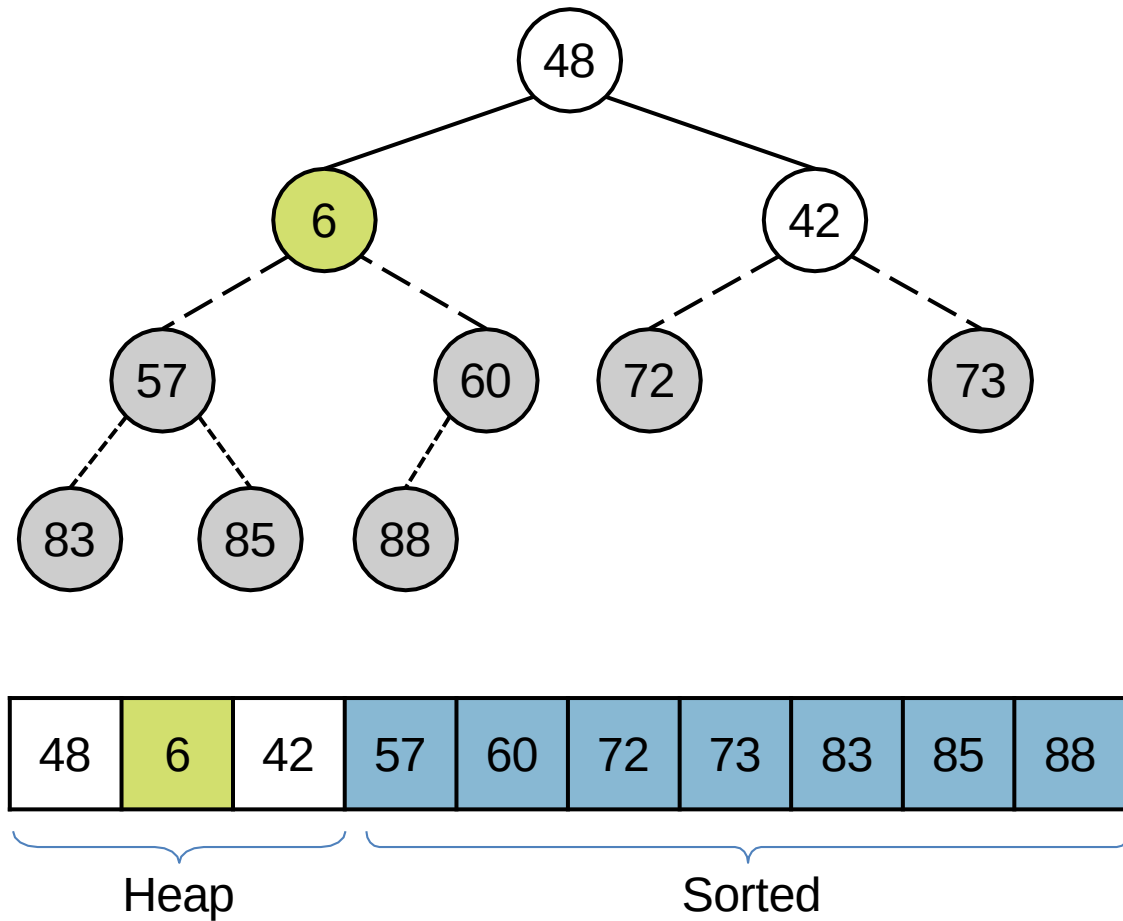


Heap Sort

- 루트 (57) 삭제

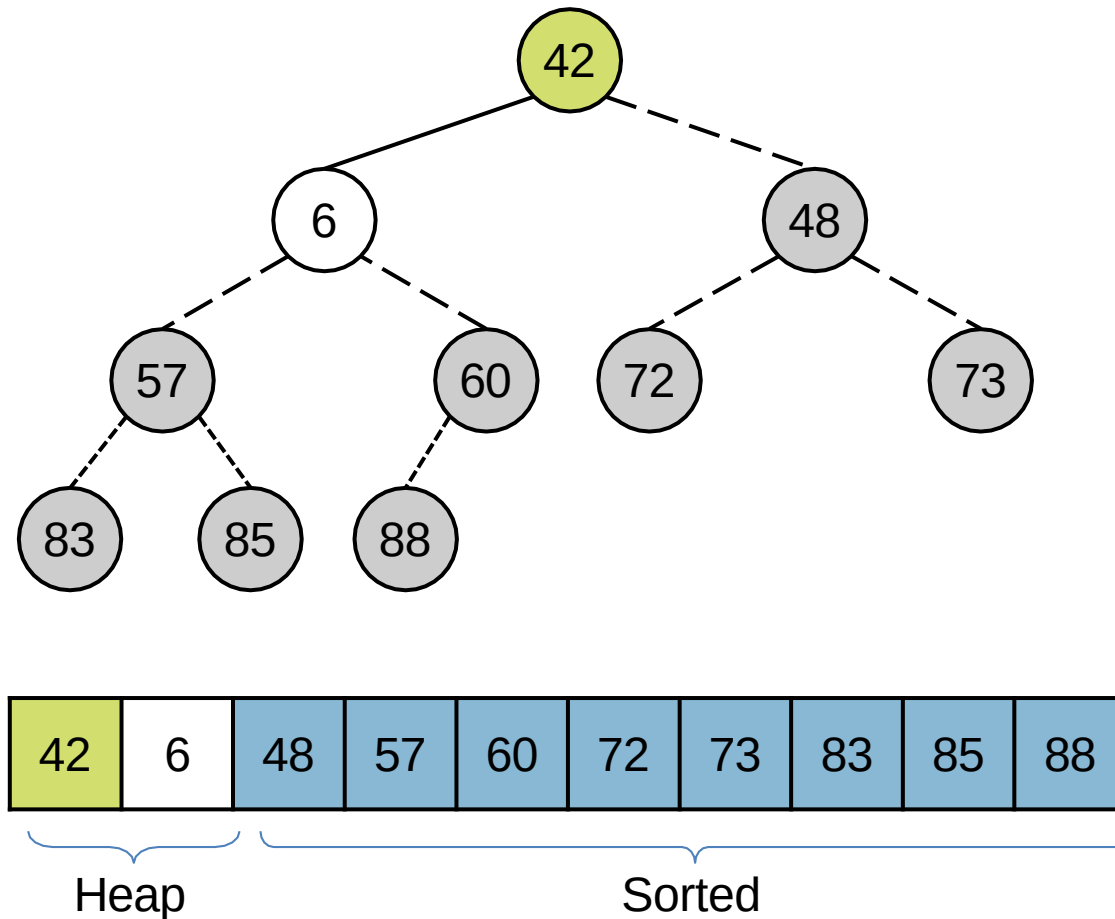


Heap Sort



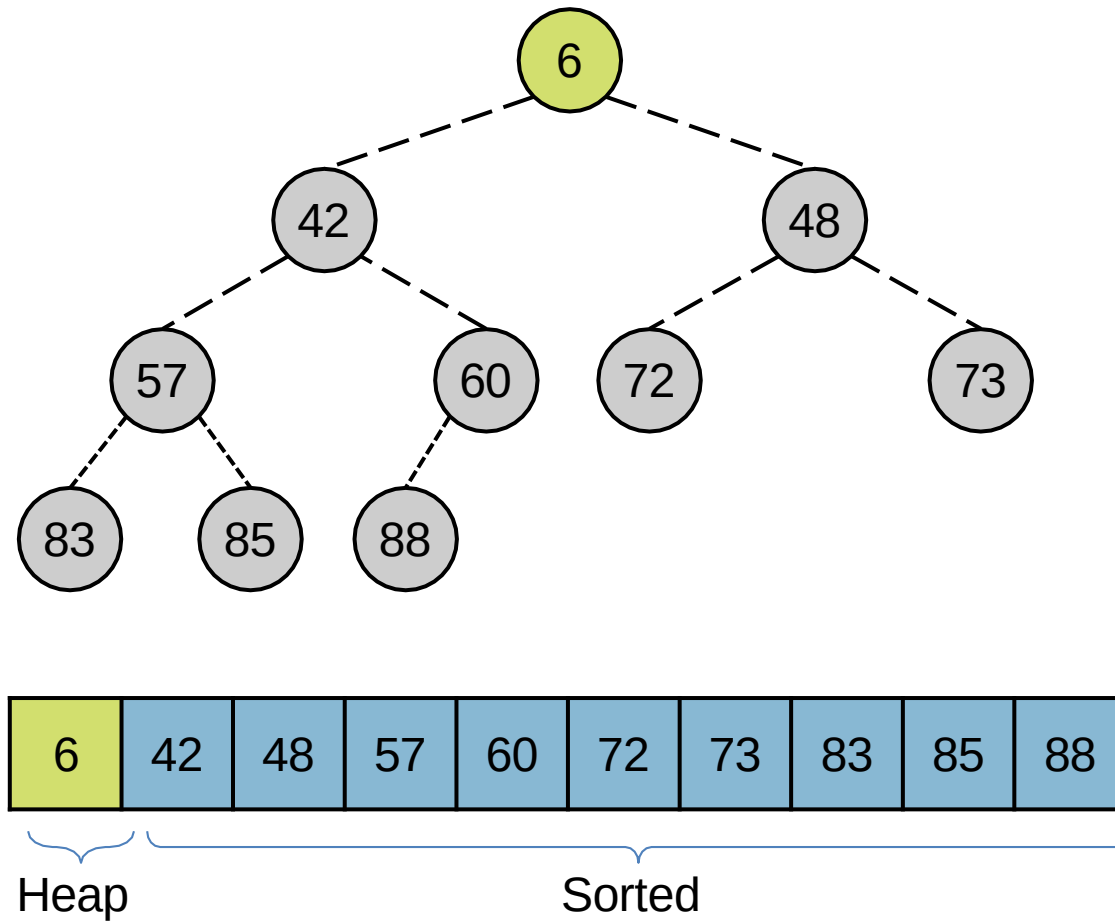
Heap Sort

- 루트 (48) 삭제



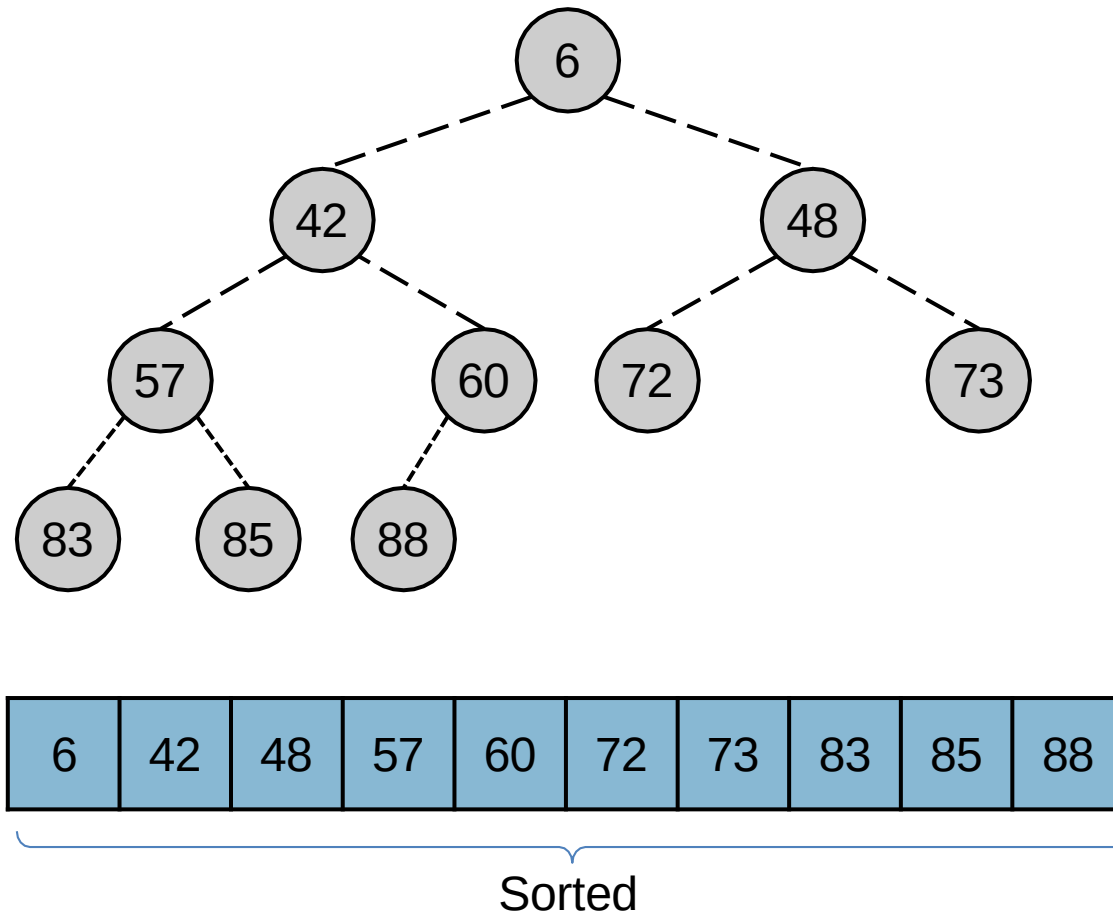
Heap Sort

- 루트 (42) 삭제



Heap Sort

- 루트 (6) 삭제 (완성)



구현

```
public void heapSort() {  
    int oldN = n;    // 정렬 후 힙 크기의 복구를 위한 변수  
  
    buildHeap();  
    for (int end = n - 1; end > 0; end--) {  
        swap(0, end);  
        n--;  
        siftDown(0);  
    }  
  
    n = oldN;  
}
```

```
private void buildHeap() {  
    for (int i = n/2 - 1; i >= 0; i--)    // 마지막 내부 노드부터 시작  
        siftDown(i);  
}
```

```
private void swap(int i, int j) {  
    int tmp = Heap[i];  
    Heap[i] = Heap[j];  
    Heap[j] = tmp;  
}
```

복잡도

- 시간복잡도

- 전체: $O(n + n \log n) \approx O(n \log n)$
 - 힙 구성: $O(n)$
 - 힙 정렬: $O(n \log n)$

- 메모리

- 추가적인 메모리 사용 없이 정렬 수행 가능(In-place 정렬 방식)

Look ahead

Search – Hashing